UNIT-III

Clustering and regression

- **Clustering** is an unsupervised learning technique in which data points are grouped into clusters such that points in the same cluster are more similar to each other than to those in other clusters.
- **Regression** is a supervised learning technique used to predict a continuous numerical value by finding the relationship between input variables and an output variable.
- Both **clustering** and **regression** are data analysis techniques used in machine learning to understand patterns in data—clustering groups similar data, while regression predicts numerical outcomes.

linear Regression

- Linear Regression is one of the most fundamental and widely used techniques in **supervised machine learning and statistics**.
- It is used to model the relationship between a **dependent (output) variable** and one or more **independent (input) variables** by fitting a linear equation to observed data.
- The main objective of linear regression is to **predict a continuous numerical value** and to **understand how input variables influence the output**.
- Linear Regression is a supervised learning technique used to **predict a continuous value** by establishing a **linear relationship** between an independent variable (input) and a dependent variable (output).
- It fits a **straight line (best-fit line)** to the given data points.

Linear Regression (Ordinary Least Squares method)

```
import pandas as pd
import numpy as np

df = pd.read_csv(r"C:\Users\sathy\OneDrive\Documents\ML AND ANIMATION III BCA AND BCA DS\ML\StudentsPerformance.csv")

x = df["reading score"].values
y = df["math score"].values

m = np.sum((x-x.mean())*(y-y.mean())) / np.sum((x-x.mean())**2)
c = y.mean() - m*x.mean()

print("Predicted Math Score:", m*98 + c)
```

Output: Predicted Math Score: 90.5694079277486

Import Libraries

- pandas → used to read the CSV file
- numpy → used for mathematical calculations

Read the Kaggle CSV Dataset

- Loads the Kaggle dataset into a DataFrame called df
- Each column in the CSV becomes a column in df

Select Input and Output Columns

- x → input feature (reading score)
- y → output label (math score)
- .values converts columns into NumPy arrays
- Simple Linear Regression studies the **relationship between one input (x)** and **one output (y)**.

Calculate Slope (m)

- Computes the **slope** of the best-fit line

Formula used:

$$m = \frac{\sum(x - \bar{x})(y - \bar{y})}{\sum(x - \bar{x})^2}$$

- This is the core **learning step**

- x.mean() → average of reading scores
- y.mean() → average of math scores
- Numerator → measures how x and y vary together
- Denominator → measures how x varies alone

For every 1-unit increase in reading score, how much the math score changes

(For every 1-mark increase in reading score, math score increases by m marks.)

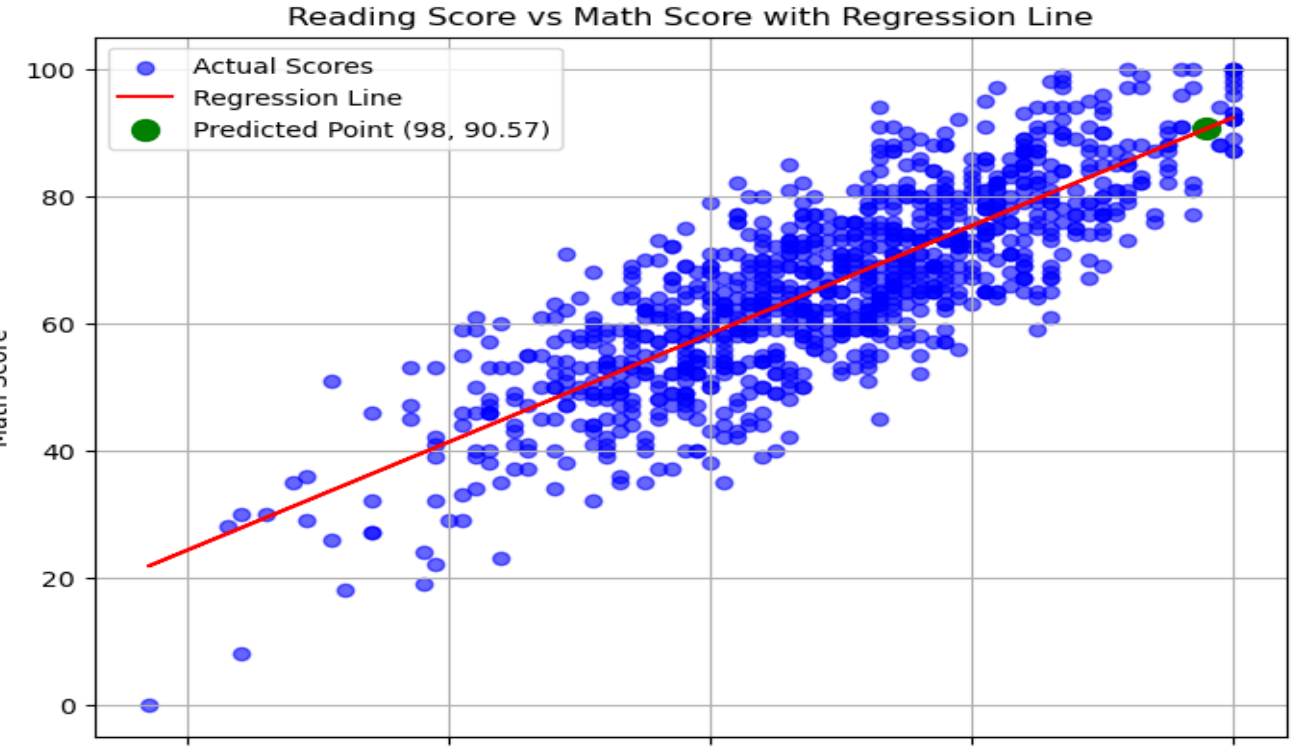
Calculate Intercept (c)

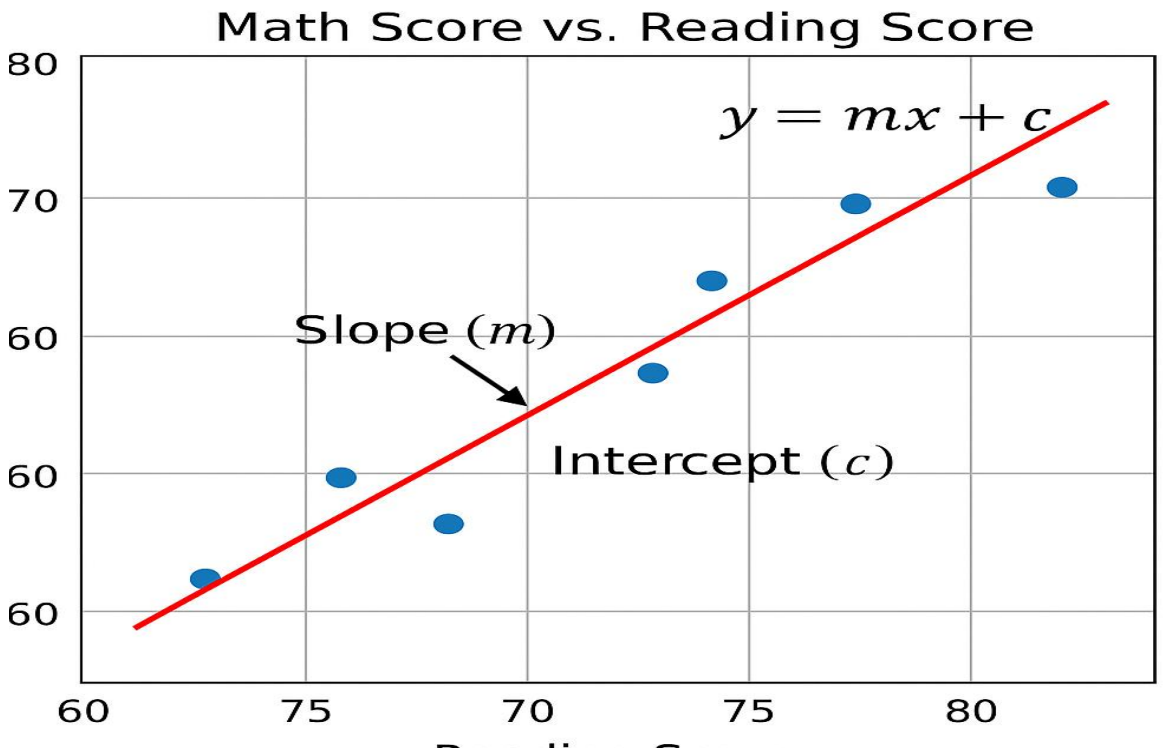
- Finds where the line crosses the y-axis
- Formula: $c = \bar{y} - m\bar{x}$

Blue points → actual scores
Red line → regression line
Green dot → predicted score for reading score 98

Make Prediction

- Uses the learned equation: $y = mx + c$
- Predicts **math score** when **reading score = 98**





Logistic Regression

- **Logistic Regression** is a statistical and machine learning method that models the probability of a binary outcome using a **logistic (sigmoid) function**, which maps predicted values to the range $[0, 1]$.

Why Logistic Regression is Needed

Linear regression cannot be directly used for classification because:

- It can produce values less than 0 or greater than 1 -Linear regression can give values like **–5 or 120**
- It does not model probabilities- Its output is just a number, not a probability
- It is sensitive to outliers-One extreme value can badly affect the decision boundary

Logistic regression overcomes these limitations by using a **non-linear activation function**.

Logistic (Sigmoid) Function

The logistic regression model uses the **sigmoid function**:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Where:

$$z = b_0 + b_1 x_1 + b_2 x_2 + \cdots + b_n x_n$$

- Output lies between **0 and 1**
- Represents probability of the positive class
- Weight: Determines the importance of each input feature.
- Bias: Adjusts the output by shifting the decision boundary.

Decision Boundary

- If probability ≥ 0.5 → Class = 1
- If probability < 0.5 → Class = 0

This threshold can be adjusted depending on the application.

Case 1: Without Bias (b = 0)

Let:

- weight $w = 1$
- bias $b = 0$

$$z = 1 \times x$$

If a student studies **0 hours**:

$$z = 0 \Rightarrow sigmoid(0) = 0.5$$

Model is **confused** (50% pass, 50% fail)

But in real life, 0 hours ≠ pass

Case 2: With Bias (b = –4)

Now:

$$z = 1 \times x - 4$$

If student studies **0 hours**:

$$z = -4 \Rightarrow sigmoid(-4) \approx 0.02$$

Strong **Fail prediction**

Logistic Regression Equation

$$z = w_1 x_1 + w_2 x_2 + b$$

Let:

- $w_1 = 2$(study hours)
- $w_2 = 0.5$(attendance)
- $b = -5$

Student A

- Study hours = **3**
- Attendance = **2**

$$z = (2 \times 3) + (0.5 \times 2) - 5$$
$$z = 6 + 1 - 5 = 2$$

High chance of **Pass**

Student B

- Study hours = **1**
- Attendance = **2**

$$z = (2 \times 1) + (0.5 \times 2) - 5$$
$$z = 2 + 1 - 5 = -2$$

High chance of **Fail**

Why Weight Matters Here

- **Study hours weight = 2** → very important
- **Attendance weight = 0.5** → less important

So, **study hours affect result more than attendance**.

Example: Student Pass / Fail Prediction

Step 1: Linear model output

Assume a logistic regression model predicts pass/fail using marks.

$$z = 0.08(Math) + 0.06(Reading) + 0.06(Writing) - 10$$

For a student with:

- Math = 70
- Reading = 65
- Writing = 60

$$z = 0.08(70) + 0.06(65) + 0.06(60) - 10$$
$$z = 5.6 + 3.9 + 3.6 - 10 = 3.1$$

Problem:
The value **3.1** is **not a probability**.

Step 2: Apply the Sigmoid function

To convert this into a probability, we use the **sigmoid function**:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\sigma(3.1) = \frac{1}{1 + e^{-3.1}} \approx 0.95$$

Step 3: Interpret the result

- Probability of **Pass** = **0.95 (95%)**
- Since 0.95 ≥ 0.5 → **Student PASSES**

The sigmoid converts the raw score into a meaningful probability.

Types of Logistic Regression

1.Binary Logistic Regression

1. Two possible outcomes (e.g., Yes/No, Pass/Fail)
2. Used when there are only two possible outcomes
3. Examples: Yes / No, Pass / Fail, Spam / Not Spam
4. Output gives the probability of one class.

2.Multinomial Logistic Regression

1. More than two unordered classes
2. Used when there are more than two classes
3. Classes are not ordered
4. Examples: Red / Blue / Green, Cat / Dog / Bird

3.Ordinal Logistic Regression

1. Ordered categories
2. Used when classes have a natural order
3. Order matters, but differences between classes are not equal
4. Examples: Low / Medium / High, Poor / Average / Good

Cost Function

Logistic regression uses **Log Loss (Cross-Entropy Loss)** instead of mean squared error:

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}[y_i \log(h_\theta(x_i)) + (1 - y_i)\log(1 - h_\theta(x_i))]$$

Case 1: Correct & Confident Prediction

- Actual $y = 1$
- Predicted $p = 0.9$

$$\text{Loss} = -\log(0.9) \approx 0.105$$

Small loss (good prediction)

Case 2: Wrong & Confident Prediction

- Actual $y = 1$
- Predicted $p = 0.1$

$$\text{Loss} = -\log(0.1) \approx 2.30$$

Huge loss (very bad prediction)

Applications of Logistic Regression

- Disease diagnosis (Yes/No)
- Spam email detection
- Credit card fraud detection
- Student pass/fail prediction
- Customer churn prediction

Advantages

- Simple and computationally efficient
- Outputs probabilities
- Easy to interpret coefficients
- Performs well for linearly separable data

Comparison with Linear Regression

Feature	Linear Regression	Logistic Regression
Output	Continuous	Binary / Categorical
Function	Linear	Sigmoid
Loss Function	Mean Squared Error	Log Loss
Use Case	Prediction	Classification

```python
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

# 1. Load Dataset
data = pd.read_csv("StudentsPerformance.csv")

# 2. Input Features
X = data[['math score', 'reading score', 'writing score']]

# 3. Output Label (PASS = 1, FAIL = 0)
y = (X.mean(axis=1) >= 50).astype(int)

# 4. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```python
# 5. Train Logistic Regression Model
model = LogisticRegression()
model.fit(X_train, y_train)

# 6. Predict ONLY ONE STUDENT
prediction = model.predict(X_test.iloc[[0]])

# 7. Display PASS / FAIL
print("STUDENT RESULT:", "PASS" if prediction[0] == 1 else "FAIL")
```

Bayesian Decision Theory

- Bayesian Decision Theory is a **fundamental framework for decision-making under uncertainty**.
- It provides a probabilistic approach to **classification and prediction** by using **Bayes' theorem** and minimizing the **expected risk (loss)**.
- It forms the theoretical foundation for many machine learning and pattern recognition algorithms.
- Bayesian decision theory is based on **Bayes' theorem**, which is given by:

$$P(\omega_i \mid x) = \frac{P(x \mid \omega_i)P(\omega_i)}{P(x)}$$

Where:
- $P(\omega_i \mid x)$ →Posterior probability
- $P(x \mid \omega_i)$ →Likelihood
- $P(\omega_i)$ →Prior probability
- $P(x)$ →Evidence

Prior Probability $P(\omega_i)$

The **prior probability** represents the probability of occurrence of a class **before observing any data**.
It reflects **existing knowledge or past experience**.

If 60 out of 100 emails are spam →Prior probability of spam = **0.6**.

Likelihood Function $P(x \mid \omega_i)$

The likelihood describes the probability of observing the feature vector x **given that the class is** ω_i.
It models the **class-conditional density** of the data.

If spam emails usually contain words like *free, offer* →The likelihood of those words given spam is high.

Posterior Probability $P(\omega_i \mid x)$

The posterior probability is the **updated probability of a class after observing the data**.
It combines both **prior knowledge and observed evidence**.

If an email has "free offer", probability of spam increases.

Evidence $P(x)$ in spam detection = Overall probability of observing that email content, regardless of spam or not spam.

Loss Function $L(\omega_i, \alpha_j)$

The loss function specifies the **cost of taking action** α_j when the true class is ω_i.
Different misclassifications may have **different consequences**.

Marking spam as normal → small problem. Marking an important mail as spam → bigger problem.

Decision Rule

The **Bayes Decision Rule** selects the action that **minimizes expected risk**:

$$R(\alpha_j \mid x) = \sum_i L(\omega_i, \alpha_j) P(\omega_i \mid x)$$

Choose the action α_j with **minimum risk**.

- The **Bayes Decision Rule** helps us choose the **best decision with the least risk (error)**.

1. R(αj|x) - Risk of taking action α_j after seeing data x.
2. $L(\omega_i, \alpha_j)$ -Loss (cost) if the true class is $\omega_i$but we choose action α_j.
3. $P(\omega_i \mid x)$ -Probability that class $\omega_i$is correct after seeing data.

- The formula calculates **expected risk (possible error cost)**.

Minimum Error Rate Classification

If a **0–1 loss function** is used, Bayesian decision theory reduces to:

$$\text{Choose } class\ \omega_i\ \ with\ maximum\ P(\omega_i \mid x)$$

This is known as the **Maximum A Posteriori (MAP) decision rule**.

- **0 loss** - If prediction is correct.
- **1 loss** - If prediction is wrong.

Choose the class with the highest posterior probability

$$\text{Choose } \omega_i\ \ with\ maximum\ P(\omega_i \mid x)$$

- $P(\omega_i \mid x)$ =Probability of class after seeing data.
- **Pick the most probable class.**

Suppose email classification:

- $P(Spam \mid x) = 0.8$
- $P(Not\ Spam \mid x) = 0.2$
- Since **0.8 is higher**, classify as **Spam**.
- This minimizes error probability.

Example

Problem:

Classify an email as **Spam (S)** or **Not Spam (N)**.

Given:

- $P(S) = 0.4$,$P(N) = 0.6$
- $P(x \mid S) = 0.7$
- $P(x \mid N) = 0.2$
- Prior probability of Spam → **P(S) = 0.4**
- Prior probability of Not Spam → **P(N) = 0.6**

These are initial beliefs before seeing the email.

- Likelihood of words in email if Spam → **P(x|S) = 0.7**
- Likelihood of words in email if Not Spam → **P(x|N) = 0.2**

This shows how well the email matches each class.

Calculate posterior probabilities:

$$P(S \mid x) \propto 0.7 \times 0.4 = 0.28$$
$$P(N \mid x) \propto 0.2 \times 0.6 = 0.12$$

Since $0.28 > 0.12$, classify the email as **Spam**.

Applications of Bayesian Decision Theory

- Pattern recognition
- Medical diagnosis
- Spam filtering
- Speech recognition
- Image classification
- Fraud detection

Classifiers

- **Classification** is a type of **supervised learning** where a model learns to put data into specific categories (classes).

Example:

- Email → Spam or Not Spam
- Student result → Pass or Fail
- Image → Cat, Dog, Bird
- The model learns from **labeled data** (data with correct answers).
- A **classifier** (classification rule) is a decision function **h(x)** that predicts a class label for a given input.
- For binary classification, classes might be {0,1} or {spam, not spam}; for multiclass, there are **more than two labels**.

In supervised classification:

- You have a **training dataset** of examples with known labels.
- The learner constructs a **model** that predicts labels for new, unseen data.
- The quality of the model is judged by how well predictions match actual labels.

Types of Classification

Binary Classification

- Only two possible class labels (e.g., yes/no, positive/negative).

Multiclass Classification

- More than two classes. Some binary methods can be extended to multiclass using strategies like one-vs-rest.

Major Classifier

A. Linear Models

- **Logistic Regression** Predicts probability of a class. Uses an S-shaped (logistic/sigmoid) curve.

Example: Spam or Not Spam prediction.

- **Linear SVM (Support Vector Machine)** – Draws a line that best separates two classes. Keeps maximum distance from both classes.

Example: Separating cats vs dogs images.

B. Distance-Based

- **k-Nearest Neighbors (k-NN)** – Looks at nearest data points. Majority class among neighbors is chosen.

Example: If 3 nearby emails are spam - New email is spam.

C. Probabilistic Models

- **Naive Bayes** – Uses probability rules (Bayes theorem). Assumes features are independent.

Example: Word probability used for spam detection.

D. Tree-based Models

- **Decision Trees** – Makes decisions like a flowchart. Splits data step by step.

Example: Age → Income → Buy product?

- **Random Forest** – ensemble of decision trees with improved accuracy (aggregates outputs).
- Many decision trees together.
- Final decision = majority vote.
- More accurate than one tree.

E. Kernel & Non-Linear Models

- **SVM with Kernel** – uses a kernel trick to handle non-linearly separable data.
- Used when data is not linearly separable. Uses special math (kernel trick).
- Example: Complex image classification.

F. Neural Classifiers

- **Perceptron** – basic linear binary classifier. Simple neural model. Works for basic binary classification.
- **Multi-layer Neural Networks** – can learn complex non-linear decision boundaries. Deep learning models. Handle complex patterns.
- Example: Face recognition, speech recognition.

How Classifiers Learn

Training a classifier generally involves:

A classifier learns from training data to predict the correct class.

1.**Estimating Parameters** using training data.

- The model learns important values (parameters) from data.
- Logistic regression learns weights.

2. Minimizing a loss (e.g., misclassification error, log-loss). Loss means error in prediction.

Misclassification error - Wrong prediction count.

Log-loss - Probability-based error.

3. Generalization — ability to correctly classify new, unseen data. Spam filter works for new emails.

Evaluation Metrics for Classification

The basic tool used for evaluating classifiers is the **confusion matrix** — a table that compares predicted vs actual class labels.

Binary Confusion Matrix components:

- After training, we check how good the classifier is.✓ Main tool: Confusion Matrix
- Actual class (true value)
- Predicted class (model output)

1. True Positive (TP)

Correct positive prediction.
Example: Spam email correctly detected.

2. True Negative (TN)

Correct negative prediction.
Example: Normal email correctly identified.

3. False Positive (FP)

Wrong positive prediction.
Example: Normal email marked as spam.

4. False Negative (FN)

Wrong negative prediction.
Example: Spam email marked as normal.

Actual \ Predicted	Positive	Negative
Positive	TP	FN
Negative	FP	TN

- TP (True Positive) – correctly predicted positive.
- TN (True Negative) – correctly predicted negative.
- FP (False Positive) – incorrect positive prediction.
- FN (False Negative) – incorrect negative prediction.

Derived Metrics

These metrics help measure how good a classifier is.

- **Accuracy** = (TP + TN) / all predictions (How many predictions are correct overall.)
- **Precision** = TP / (TP + FP) (Out of predicted positives, how many are correct.)
- If 20 emails predicted spam but only 15 actually spam →Precision = 15/20.
- **Recall (Sensitivity)** = TP / (TP + FN).(Out of actual positives, how many detected correctly.)
- If 25 spam emails exist but model finds only 20 → Recall = 20/25.
- **F1-Score** = harmonic mean of Precision and Recall.
- Average of Precision and Recall. Used when both precision and recall matter.
- Simply: Balance between precision and recall.

Decision Boundaries

- A **decision boundary** is a line or curve that separates classes.
- It shows how the model divides data.

Linear Boundary

- Straight line (or flat surface).
- Examples: Logistic Regression, Linear SVM

Used when data is simple.

Non-Linear Boundary

- Curved or complex boundary.

Examples:

- Decision Trees
- Kernel SVM
- Neural Networks

Used when data is complex.

Bayes' Theorem Formula

$$P(\omega_i \mid x) = \frac{P(x \mid \omega_i)\, P(\omega_i)}{P(x)}$$

Where:

- $P(\omega_i \mid x)$ →**Posterior probability** (probability of class after seeing data)
- $P(x \mid \omega_i)$ →**Likelihood** (probability of data given class)
- $P(\omega_i)$ →**Prior probability** (probability of class before data)
- $P(x)$ →**Evidence** (total probability of data)

Spam Email Classification

Assume:

- $P(Spam) = 0.4$ →Prior probability of spam
- $P(Not\ Spam) = 0.6$ →Prior probability of non-spam
- $P("Offer" \mid Spam) = 0.7$ →Likelihood
- $P("Offer" \mid Not\ Spam) = 0.2$

We receive an email containing **"Offer"** word.

We must calculate probability it is spam.

Calculate Evidence $P(x)$

Evidence is total probability of observing the word "Offer":

$$P(x) = P(x \mid Spam)P(Spam) + P(x \mid Not\,Spam)P(Not\,Spam)$$

Substitute values:

$$P(x) = (0.7)(0.4) + (0.2)(0.6)$$

$$P(x) = 0.28 + 0.12 = 0.40$$

Calculate Posterior Probability (Spam)

$$P(Spam \mid x) = \frac{P(x \mid Spam)P(Spam)}{P(x)}$$

Substitute:

$$P(Spam \mid x) = \frac{(0.7)(0.4)}{0.40}$$

$$P(Spam \mid x) = \frac{0.28}{0.40}$$

$$P(Spam \mid x) = 0.7$$

Posterior probability of spam = **0.7 (70%)**

Calculate Posterior Probability (Not Spam)

$$P(Not\ Spam \mid x) = \frac{(0.2)(0.6)}{0.40}$$

$$P(Not\ Spam \mid x) = \frac{0.12}{0.40}$$

$$P(Not\ Spam \mid x) = 0.3$$

Posterior probability = **0.3 (30%)**

Decision Rule (Bayesian Decision)

Choose class with highest posterior probability:

$$P(Spam \mid x) = 0.7 > 0.3$$

Final Decision: **Email is Spam**

Expected Risk (Loss) Calculation

Bayesian decision theory minimizes expected loss:

$$R(\alpha_i \mid x) = \sum_j \lambda\left(\alpha_i \mid \omega_j\right) P\left(\omega_j \mid x\right)$$

Where:

- $R(\alpha_i \mid x)$ →Risk of decision
- λ→ Loss function
- Choose decision with minimum risk.

Example:

If misclassification loss = 1:

Risk(Spam decision):

$$R = 1 \times P(Not\,Spam \mid x) = 1(0.3) = 0.3$$

Risk(Not Spam decision):

$$R = 1 \times P(Spam \mid x) = 1(0.7) = 0.7$$

Minimum risk = 0.3 → Choose Spam.

Discriminant functions

- A **discriminant function** in machine learning and statistics is a **function of input features that produces a score used to decide a class label**.
- It maps feature vectors **x** into a discriminant score for each class, and the highest score determines the predicted class.

In general form for linear discriminant functions:

$$g_i(\mathbf{x}) = w_{i0} + w_{i1}x_1 + w_{i2}x_2 + \cdots + w_{in}x_n$$

Here, $g_i(\mathbf{x})$is the discriminant function value for class **i**, $x_j$are features, and $w_{ij}$are learned weights.

Suppose we classify students as Pass or Fail:

Feature:

- x_1 = Marks

Discriminant function:

$$g(x) = 5 + 0.8 \times marks$$

If:

- Marks = 60 - Score = 5 + 48 = **53**
- Marks = 30 - Score = 5 + 24 = **29**

Higher score - More chance of **Pass**

Purpose in Classification

The purpose of a discriminant function is to **separate classes** by assigning each new example to the class with the highest discriminant score:

- For **binary classification**, compare two discriminant values:

$$\text{Predict } \textit{class 1 if } g_1(\mathbf{x}) \geq g_2(\mathbf{x})$$

- For **multiclass problems**, choose class **k** such that:

$$g_k(\mathbf{x}) = \max_i \quad g_i(\mathbf{x})$$

Thus, discriminant functions define **decision boundaries** in feature space.

A **decision boundary** is a line or curve that:

- Separates different classes
- Shows where one class changes to another

Example:

- Marks above 50 → Pass
- Marks below 50 → Fail

Here **50 marks is the decision boundary**.

Linear Discriminant Functions

Definition

- A **Linear Discriminant Function (LDF)** is a formula used to **separate two or more classes** using a straight line

$$g(\mathbf{x}) = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_n x_n$$

- It combines input features with weights to produce a score.
- Based on the score, we decide the class.

Relationship to Classification

For two-class problems:

- The hyperplane $g(\mathbf{x}) = 0$ acts as the decision boundary.
- If $g(\mathbf{x}) > 0$,assign class 1; if $g(\mathbf{x}) < 0$,assign class 2.

Example

If features are x_1, x_2 ,a linear discriminant function might be:

$$g(\mathbf{x}) = 0.5 + 1.2x_1 - 0.8x_2$$

This yields a **line** separating classes where $g(\mathbf{x}) = 0$

Discriminant Function Analysis

Discriminant Function Analysis (DFA) or **Linear Discriminant Analysis (LDA)** is a method used to:

- Separate different classes clearly
- Find the best line or direction to distinguish classes
- It calculates weights so the classes become **easier to separate**.

How It Works

- LDA constructs a **linear discriminant function**, which is a weighted combination of input features.
- The weights are chosen so that:
  - The distance between different classes is maximized.
  - The variation within each class is minimized.

This results in better separation of classes for classification.

Projection Concept

LDA projects high-dimensional data onto a **lower-dimensional space (usually a line or subspace)** where:

- Samples from the same class remain close together.
- Samples from different classes are well separated.

This improves classification performance.

Fisher's Linear Discriminant

Fisher proposed selecting a projection direction that:

- Maximizes **between-class scatter** (distance between class means).
- Minimizes **within-class scatter** (spread within each class).

This produces the direction where class separation is strongest.

Decision Boundaries

Linear discriminant functions generate **linear decision boundaries** that separate different classes in feature space.

- In **two-dimensional data**, the boundary is a straight line.
- In **three dimensions**, it becomes a plane.
- In higher dimensions, it is called a **hyperplane**.

However, in some classification problems, class separation is complex. In such cases, **nonlinear discriminant functions** (such as polynomial or kernel-based functions) are used to produce curved or flexible boundaries.

Extensions and Variants

Quadratic Discriminant Analysis (QDA)

- Quadratic Discriminant Analysis is an extension of LDA where each class has its own covariance matrix.
- This allows the decision boundary to be curved (quadratic) instead of linear, which can better model complex class distributions.

Flexible Discriminant Analysis

- Flexible Discriminant Analysis incorporates nonlinear transformations, such as splines or other basis functions.
- This improves class separation when linear boundaries are not sufficient.

Regularized Discriminant Analysis

- Regularized Discriminant Analysis introduces regularization into covariance estimation.
- This helps reduce overfitting, especially when data is limited or highly variable, leading to more stable classification performance.

Properties and Interpretation

Number of Discriminant Functions

- In multiclass classification with **k classes**, a maximum of **k – 1 discriminant functions** can be derived.
- Each discriminant function represents a direction in which the classes show the greatest separation.

4 classes (e.g., four types of patients)

Then maximum discriminant functions:

$$4 - 1 = 3$$

These three functions help distinguish all four groups clearly.

Dimensionality Reduction

- Discriminant functions can also be used for **dimensionality reduction**.
- They transform high-dimensional data into a lower-dimensional space while preserving class discrimination, which improves classification efficiency and visualization.
- A discriminant function combines these features into a **single new variable (discriminant score)**:
- Instead of analyzing 5 features separately,(Math marksScience marksEnglish marksAttendanceAssignment scores)
- We use **one combined score** that best separates Pass and Fail students.

Use Cases of Discriminant Functions in Machine Learning

Supervised Classification

Discriminant functions are commonly used in supervised learning tasks where labeled data is available and clear class separation is required.

Dimensionality Reduction Before Classification

They help reduce the number of features while maintaining essential class-related information, thereby improving model performance and reducing computational complexity.

Pattern Recognition Applications

Discriminant functions are widely applied in pattern recognition problems such as:

- Face recognition
- Medical diagnosis
- Speech and image classification

These applications rely on effective discrimination between different groups or categories.